PROJECT NAME :AI STUDY BUDDY API

TEAM ID : SWTID-2026-8711

TEAM LEADER :

SHEBANIAH S : shebaniah2007ss@gmail.com

TEAM MEMBER :

ABUROOBA R : aaburooba@gmail.com

ARCHANA R : archa789na@gmail.com

DHIVYA J : jdhivya358@gmail.com

AI Study Buddy API – Intelligent Learning and Study Support

Project Description

AI Study Buddy API is an AI-powered learning support system built to make studying more structured, quicker, and more engaging. The platform combines modern web development methods with Generative AI to help learners turn their study materials into handy revision resources. Using secure REST APIs, students can send their learning content and ask for AI-generated summaries, flashcards, practice quizzes, and personalized study schedules. The backend is developed with Node.js and Express.js, and MongoDB provides permanent NoSQL data storage. Mongoose supplies schema-based data handling and validation. JWT authentication and role-based access control safeguard private resources, and bcryptjs is used to protect saved passwords.

One key strength of AI Study Buddy API is its link with the Google Gemini AI service. The backend passes the relevant learning content and structured prompts to the model and gets back freshly generated educational answers. This approach lets the system build study resources from the learner's actual material rather than relying only on fixed or ready-made templates.

Scenario-Based Case Study

Background

Take a student named Rahul who is getting ready for semester examinations while also building technical skills for internships and future job placements. His study resources are scattered across class notes, PDF documents, textbooks, online references, and handwritten

material. Since the information is divided among many sources, organising revision becomes hard, especially when examination dates are close. Rahul needs an easy way to find key concepts, prepare quick revision material, practise questions, and plan his free study hours.

Traditional preparation methods force students to read long documents by hand, prepare notes, write practice questions, create flashcards, and build revision timetables. These repeated tasks use up valuable study time and can make preparation uneven. A combined AI-based solution can cut down this routine work while leaving the learner in control of the final study material.

Problem Statement

During examination preparation, Rahul faces the following practical problems:

- Understanding long academic content in a short preparation time is difficult.
- Making and rewriting notes by hand takes away a lot of revision time.
- Creating flashcards and practice questions again and again needs extra effort.
- A proper study timetable is hard to make when free time and examination dates keep changing.
- Learning resources created in different study sessions are not kept in one safe place.
- Self-assessment is hard when students must prepare every practice quiz on their own.
- Using separate applications for notes, revision questions, quizzes, and scheduling breaks the flow of studying.

Proposed Solution

AI Study Buddy API offers a single, central learning-support backend that automates common academic preparation tasks through Generative AI. After authentication, a student can send study content and choose the learning operation needed. The backend validates the request, contacts the AI service when required, saves useful resources in MongoDB, and sends back structured results in REST API responses.

The backend receives requests through protected REST endpoints and performs the following tasks:

- Confirms the user's identity through JWT-based authentication.
- Keeps user profiles and learning resources in MongoDB.
- Sends suitable study content to Google Gemini through a dedicated AI service module.
- Creates short, easy-to-read summaries for quick revision.
- Makes question-and-answer flashcards for active recall practice.
- Generates multiple-choice questions to check knowledge.
- Builds study schedules based on learning goals, available time, and examination dates.

- Saves generated resources so students can come back to them in later revision sessions.

The separation of authentication, application logic, database access, file processing, and AI communication keeps the backend well organised. This modular design makes testing simpler and lets new services be added without redesigning the whole application.

System Usage

Student Usage

Students first register an account and log in to the platform. After authentication is done, they can upload study material or type in text and pick an AI operation such as summary generation, flashcard creation, quiz generation, or study-plan preparation. Results are displayed in the application and can be saved for later revision.

Administrator Usage

Administrators monitor the platform and help keep the system running smoothly. Their tasks may include handling user accounts, reviewing application activity, watching AI-service usage, checking system logs, and protecting the overall integrity of the backend.

Expected Outcome

Bringing Generative AI together with a protected backend can make academic preparation more effective. Rather than spending most of their time turning raw study material into revision resources, students can use automatic generation to get a first set of summaries, flashcards, quizzes, and schedules, then check and use those resources in their preparation.

JWT security, MongoDB storage, Mongoose validation, and Gemini integration form a well-organised technical base for the system. Since the application is split into independent modules, each component can be tested and improved without needlessly affecting the other services.

Hence, AI Study Buddy API is designed as a scalable educational support platform that helps students with revision, self-assessment, and time planning through one API-driven workflow.

Software Requirements

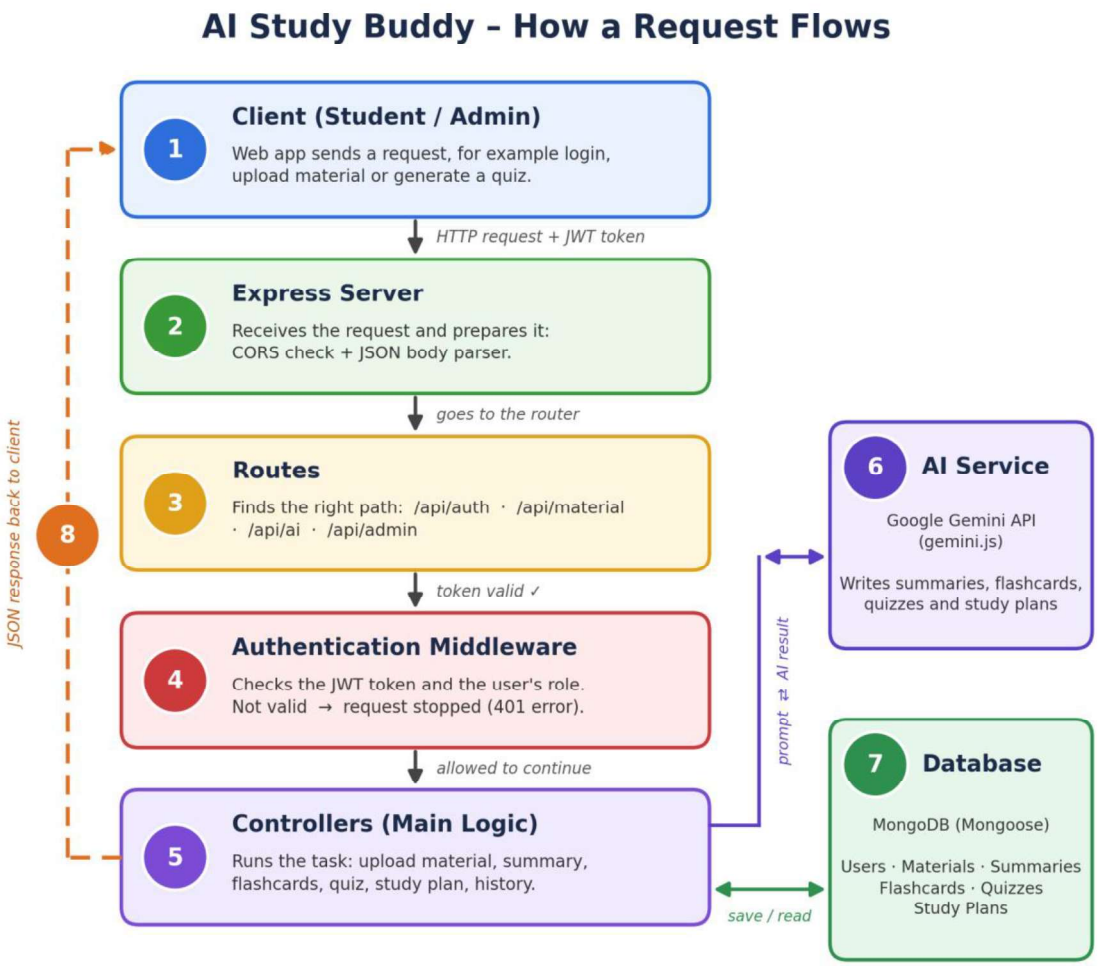
- **Operating System: Windows 10/11, macOS, or Linux for development and running on any platform.**
- **Node.js (v16 or later): JavaScript runtime used to run the backend application and server-side logic.**
- **npm (v8 or later): Package manager used to install and manage project dependencies.**
- **Express.js: Web framework used to build REST API routes, middleware, and request handling.**
- **MongoDB: NoSQL database used to store accounts, study materials, generated resources, and related application data.**
- **Postman: API testing tool used to send requests and check response data, authentication, and endpoint behaviour.**

- **Code Editor: Visual Studio Code or any other suitable development IDE.**

Hardware Requirements

- **Processor: Intel Core i5 (8th Generation or later), AMD Ryzen 5, or a similar processor.**
- **RAM: Minimum 8 GB; 16 GB is better when running the backend, database, development tools, and AI-related processes together.**
- **Storage: At least 1 GB of free space for source files, packages, logs, and local project data.**

Epic 1: System Architecture



Follow the numbers 1 → 8. Students and admins use the same path; the role check happens in step 4.

AI Study Buddy API uses a modular REST-based architecture to deliver secure learning services and AI-assisted content generation. Express.js receives client requests, JWT middleware checks the protected routes, MongoDB handles permanent data, and AI requests are passed on through a dedicated Gemini service.

The application splits authentication, routing, business rules, AI communication, and database operations into separate modules. This structure makes the code easier to read, makes problems easier to find, and gives a clear way to grow in the future.

System Components

• Client Application: A React-based frontend can offer registration, login, study-material submission, and use of the summary, quiz, flashcard, and study-plan functions through the REST API.

• Express Server: The Express layer takes in HTTP requests, applies middleware, picks the correct route, calls the required logic, and sends back a suitable response.

• Authentication Middleware: JWT middleware verifies access tokens, recognises the current user, guards private endpoints, and enforces role restrictions where needed.

• Route Handlers: API routes group the operations for authentication, material management, AI generation, and administrator functions.

• AI Service Layer: A separate service talks to Google Gemini and builds the requests for summaries, flashcards, quizzes, and personalized study plans.

• Database Layer: MongoDB and Mongoose keep account details, uploaded learning material, and generated study resources with validation and structured access.

The data model of AI Study Buddy API is built around related MongoDB collections. User accounts are connected to study materials and the AI-generated resources produced from those materials. These logical links help the application fetch a student's information and learning resources efficiently.

Relationship Summary

- **User → Study Material (One-to-Many):**One User can submit many Study Materials.
- **Study Material → Summary (One-to-One):**Each Study Material can have an associated AI-generated Summary.
- **Study Material → Flashcards (One-to-Many):**A single Study Material can produce several Flashcards.
- **Study Material → Quiz (One-to-One):**A Study Material can be used to create a Quiz containing multiple questions.
- **User → Study Plan (One-to-Many):**One User can maintain multiple Study Plans for different subjects or examination schedules.

- **User → AI Resources (One-to-Many):**All generated learning resources remain linked to the user who requested them, supporting organized retrieval.

Major Features

AI Study Buddy API provides learning assistance through protected and scalable REST endpoints. The main functions are:

- **Secure User Authentication: JWT tokens protect private operations, while bcrypt-based password hashing protects stored credentials.**
- **Study Material Management: Students can upload and manage learning content, which becomes the input for AI-supported processing.**
- **AI Summary Generation: The system asks the Gemini service for short, student-friendly summaries based on the given study material.**
- **AI Flashcard Generation: The application turns important concepts into question-and-answer cards that support active recall.**
- **AI Quiz Generation: The service builds multiple-choice practice questions from the given content for self-testing.**
- **AI Study Plan Generation: The platform builds a study schedule based on learning targets, free time, and examination needs.**
- **Role-Based Access Control (RBAC): Student and administrator permissions are kept separate so that management functions stay restricted.**
- **MongoDB Data Storage: User records, learning inputs, and generated educational resources are saved in MongoDB for later use.**

Roles and Responsibilities

Student

Students are the primary users of the platform. They can register an account, log in securely, upload study materials, ask for AI-generated summaries, flashcards, quizzes, and study plans, and open previously generated resources.

Administrator

Administrators look after the application environment. They can manage registered users, watch system activity, review usage, protect platform integrity, and carry out the administrative work needed for dependable service.

AI Study Buddy API follows the Model-View-Controller pattern to keep data operations, application logic, and API communication clearly apart. This arrangement supports maintainability and makes each component easier to test and extend.

Model Layer (Data Layer)

The Model layer describes the MongoDB collections using Mongoose schemas. It handles user records, submitted study material, and generated resources while applying validation rules and database operations.

Epic 2: Project Setup and Configuration

Creating the Project Directory

1. Create a new project folder named AI Study Buddy API.
2. Open the project directory in Visual Studio Code.
3. Initialize the Node.js project and install the packages required by the backend.

Project Initialization

Open the VS Code terminal inside the project folder and run:

```
npm init -y
```

Install the project dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

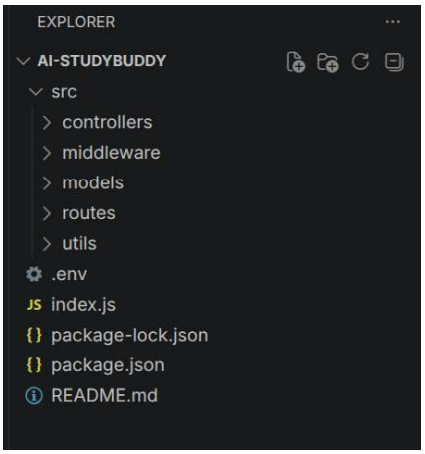
Add Nodemon as a development dependency:

```
npm install --save-dev nodemon
```

Create the following project files:

- index.js
- .env
- package.json
- .gitignore

Create the required folder structure:



Environment Configuration

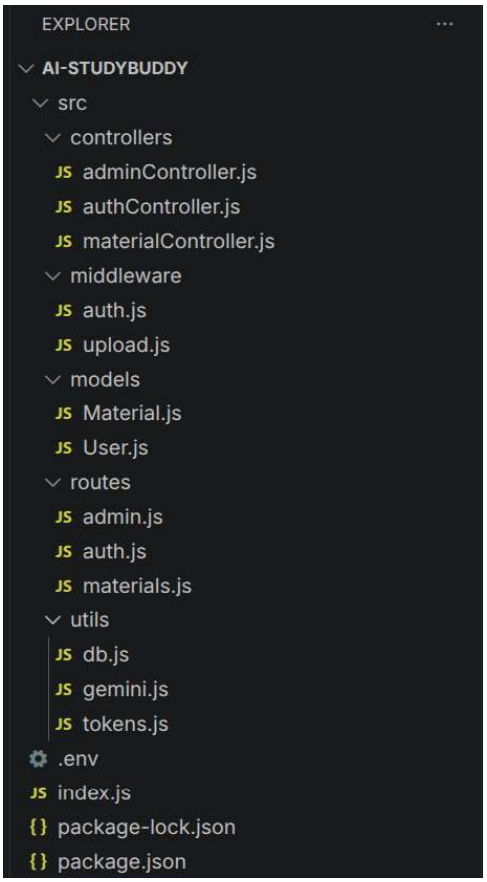
Create a .env file in the project root and add the environment values required by the server.

PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key

Epic 3: Backend Development

The AI Study Buddy API backend is built with Node.js, Express.js, and MongoDB using a modular server structure. Routes, controllers, middleware, models, and utility services are kept apart so that authentication, learning-material handling, AI generation, and administration can develop independently. The backend provides protected REST endpoints for these operations.

Backend Structure



Core Backend Script Implementations

1. Application Entry Point (index.js)

index.js is where the server starts. It loads environment settings, creates the Express application, sets up middleware, connects to MongoDB, registers the API routes, and starts listening for incoming requests once the database connection is ready.

```
JS index.js > ...
1  require("dotenv").config();
2  require("express-async-errors");
3
4  const express = require("express");
5  const cors = require("cors");
6  const connectDB = require("./src/utils/db");
7
8  const app = express();
9
10 // Middleware
11 app.use(express.json());
12 app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));
13
14 // Routes
15 app.use("/api/auth", require("./src/routes/auth"));
16 app.use("/api/materials", require("./src/routes/materials"));
17 app.use("/api/admin", require("./src/routes/admin"));
18
19 // Health check
20 app.get("/", (req, res) => res.json({ message: "AI StudyBuddy API is running" }));
21
22 // Global error handler
23 app.use((err, req, res, next) => {
24   console.error(err.message);
25   const status = err.status || 500;
26   res.status(status).json({ message: err.message || "Something went wrong" });
27 });
28
29 const PORT = process.env.PORT || 5000;
30
31 connectDB().then(() => {
32   app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
33 });
34
```

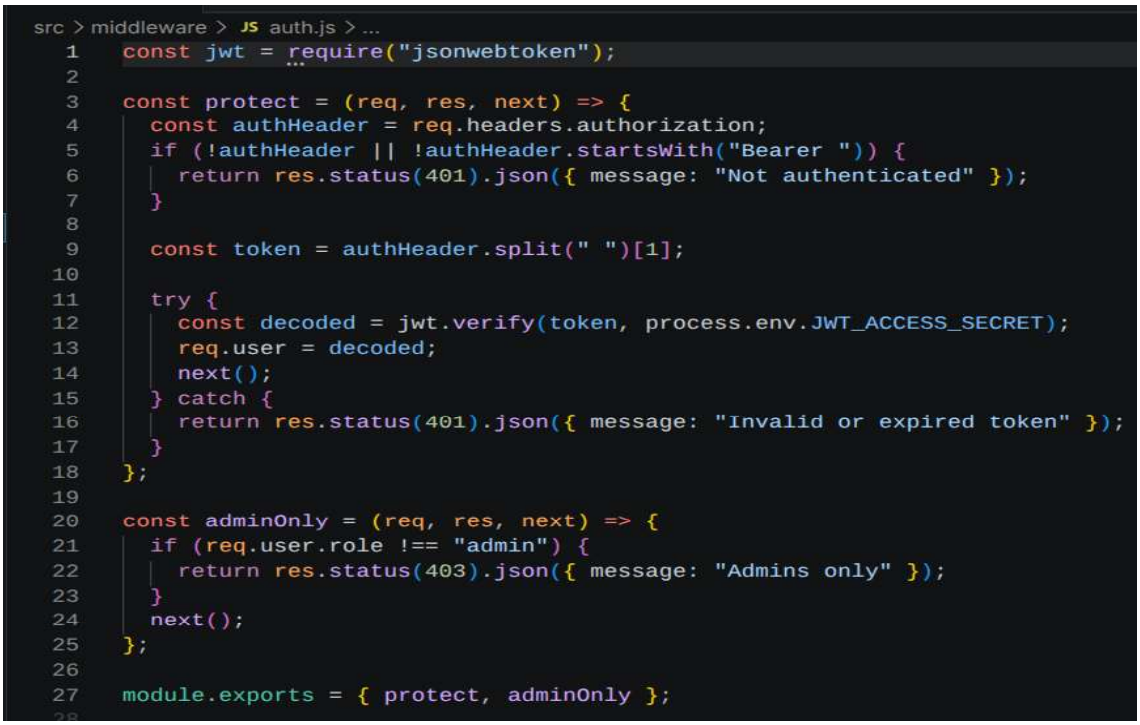
2. Database Connection (src/utils/db.js)

This utility sets up the MongoDB connection through Mongoose. The connection is made at application startup so that requests needing the database are handled only when the data service is ready.

```
src > utils > JS db.js > ...
1  const mongoose = require("mongoose");
2  const dotenv = require("dotenv");
3
4  dotenv.config();
5
6  const connectDB = async () => {
7    const conn = await mongoose.connect(process.env.MONGO_URI);
8    console.log(`MongoDB connected: ${conn.connection.host}`);
9  };
10
11 module.exports = connectDB;
12
```

3. Authentication Middleware (src/middleware/auth.js)

The authentication middleware verifies JWT access tokens on protected endpoints. It recognises the logged-in user and can enforce role-based restrictions so that administrator operations are not open to ordinary student accounts.



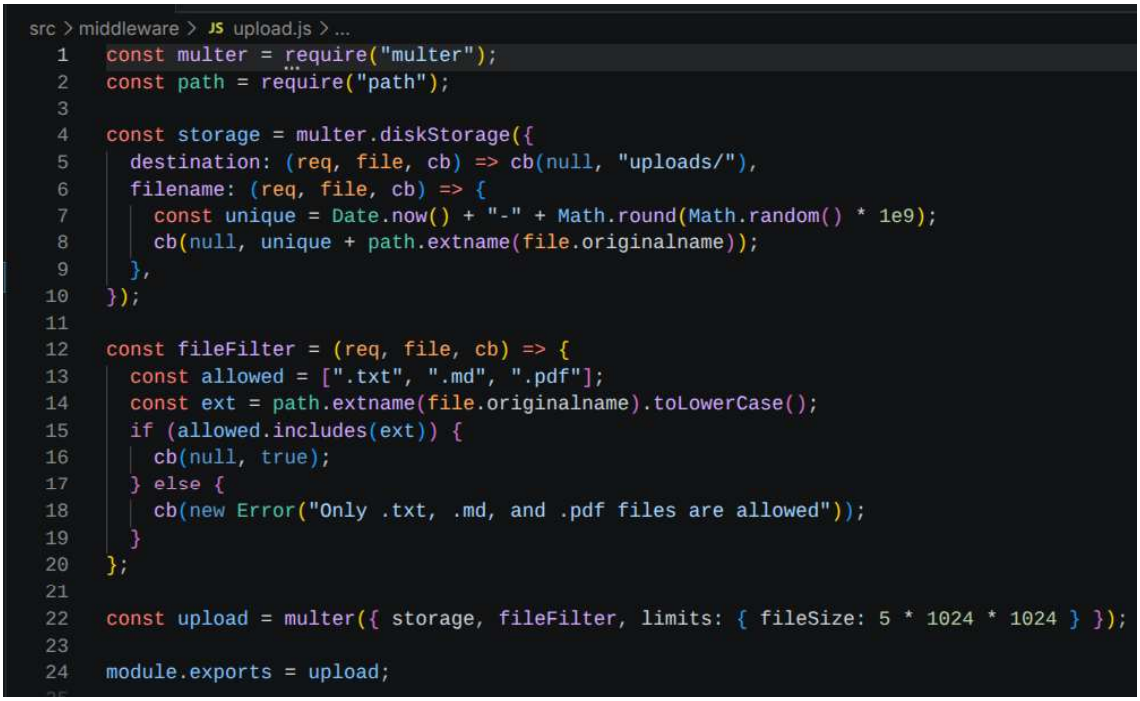
4. AI Integration (src/utils/gemini.js)

This utility connects the backend with the Google Gemini 2.5 Flash service. It builds prompts, sends learning content to the model, and gives back generated material for summaries, flashcards, quizzes, and study plans.



5. File Upload Middleware (src/middleware/upload.js)

The upload middleware accepts study-material files and does the necessary validation before the files or extracted content move on to the next processing stages.



Backend Development Demonstration:

AI_Studybuddy_Backend_Demo.mkvAI_Study_Buddy_API_Backend_Demo.mkv

Epic 4: Database Configuration

AI Study Buddy API uses MongoDB as the main NoSQL data store and Mongoose as the ODM layer. MongoDB stores account information, study materials, and generated learning resources, while Mongoose supplies schemas, validation, and easy query operations.

MongoDB Configuration

Install Mongoose with the following command:

```
npm install mongoose
```

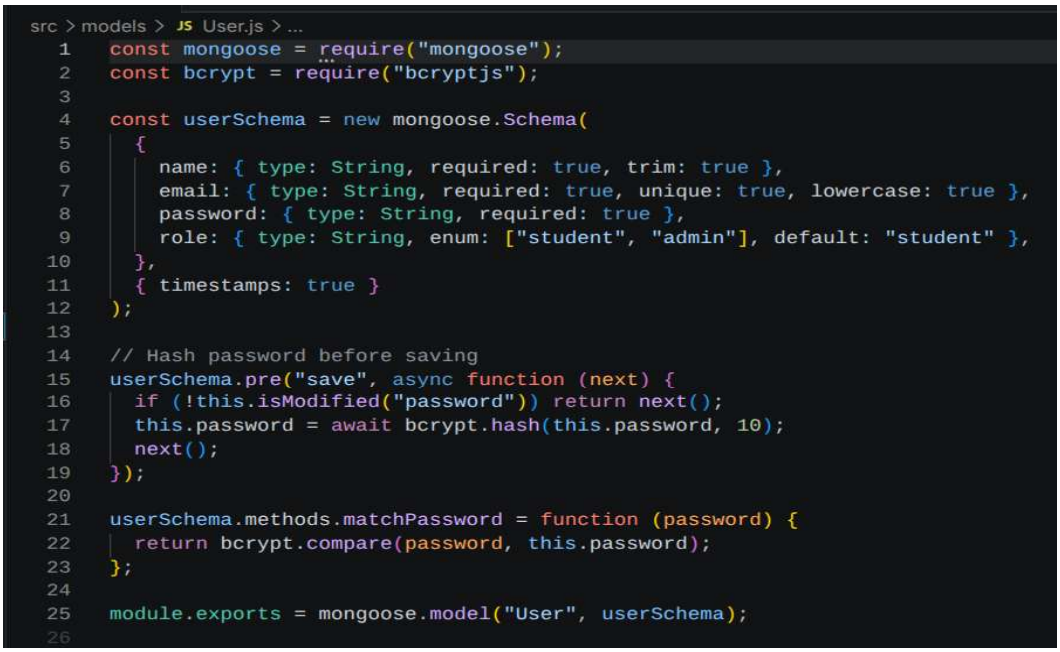
Establish Database Connection

Before the application performs database operations, it must connect to MongoDB through Mongoose. The connection utility is called at server startup to give the API a reliable data-access layer.

Mongoose Schema Definitions

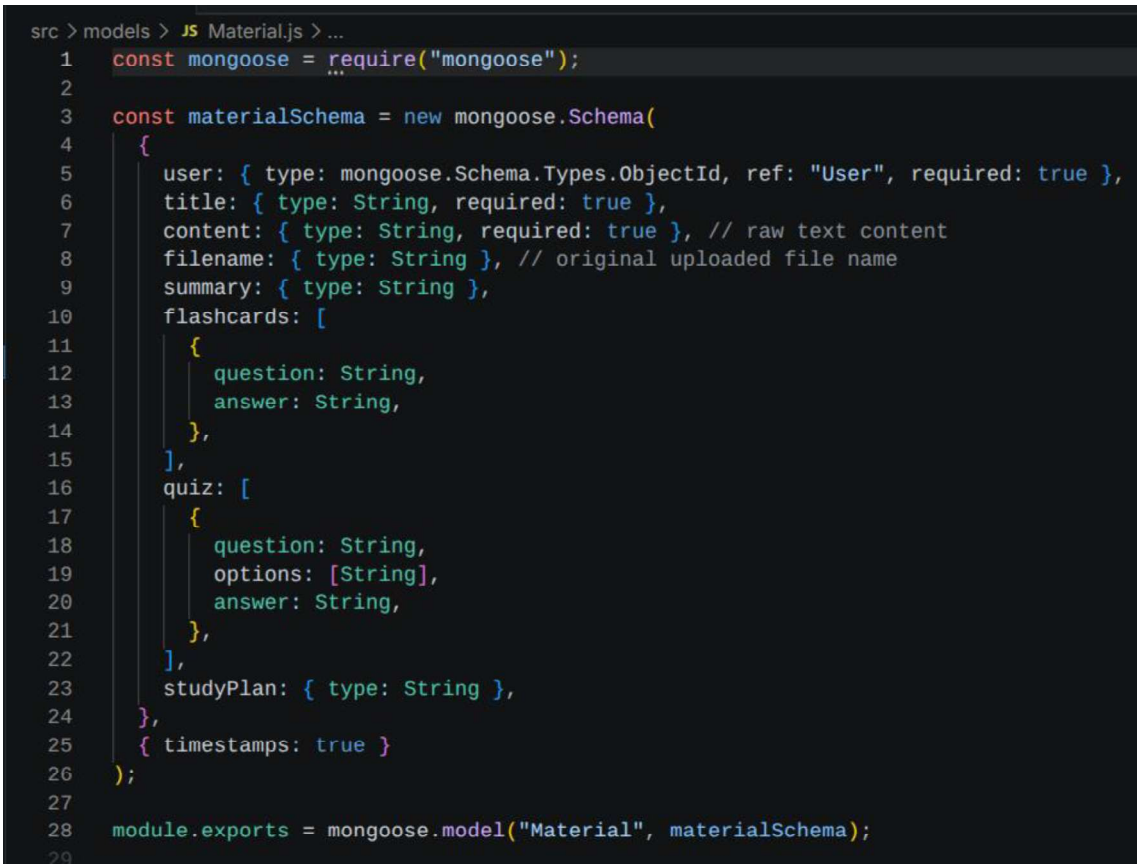
1. User Schema (models/User.js)User.js

The User schema holds authentication and profile information for registered accounts. It contains fields such as name, email, password, and role. Passwords are hashed before saving so that the original credentials are never stored as plain text.



2. Study Material Schema (models/Material.js)

The Study Material schema holds the content submitted by users. This content is the source for AI operations such as summary creation, flashcard generation, quiz preparation, and study-plan support.



Database Collections

The AI Study Buddy API database has collections for the main learning entities used by the application:

- **Users – Keeps account, authentication, and role information.**
- **Study Materials – Keeps the learning content submitted for processing.**

API Route Interface Endpoints

The backend provides RESTful endpoints that link the client application with authentication, database functions, and AI services.

1. Authentication APIs

Provides registration, login, JWT token handling, and protected-route access.

2. Study Material APIs

Allows uploading, fetching, updating, and managing student study materials.

3. AI Feature APIs

Provides endpoints for the following learning operations:

- AI Summary Generation
- AI Flashcard Generation
- AI Quiz Generation
- AI Study Plan Generation

4. Administrative APIs

Gives restricted endpoints for administrator tasks such as user management and monitoring of application operations.

Epic 5: Project Execution

The AI Study Buddy API backend can be run locally with Node.js. Before starting the application, MongoDB must be running and the required environment variables must be set. Postman or Thunder Client can then be used to test the REST endpoints and their responses.

Running the Backend Server

Install all project dependencies:

```
npm install
```

Launch the development server:

```
npm start
```

After startup, the application connects to MongoDB and begins accepting API requests on the configured port.

API Testing

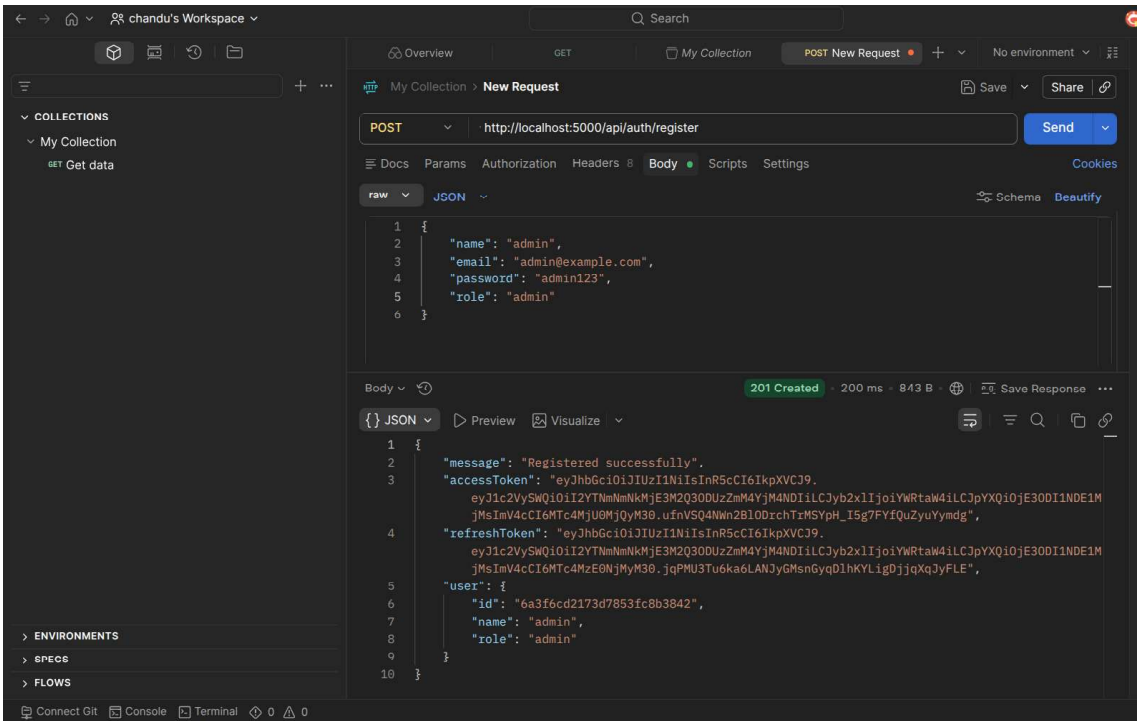
Use Postman to test the backend endpoints and inspect request and response data.

1. User Registration

Creates a new account from the given registration details and saves the user's password in hashed form inside MongoDB.

Endpoint:

POST /api/auth/register

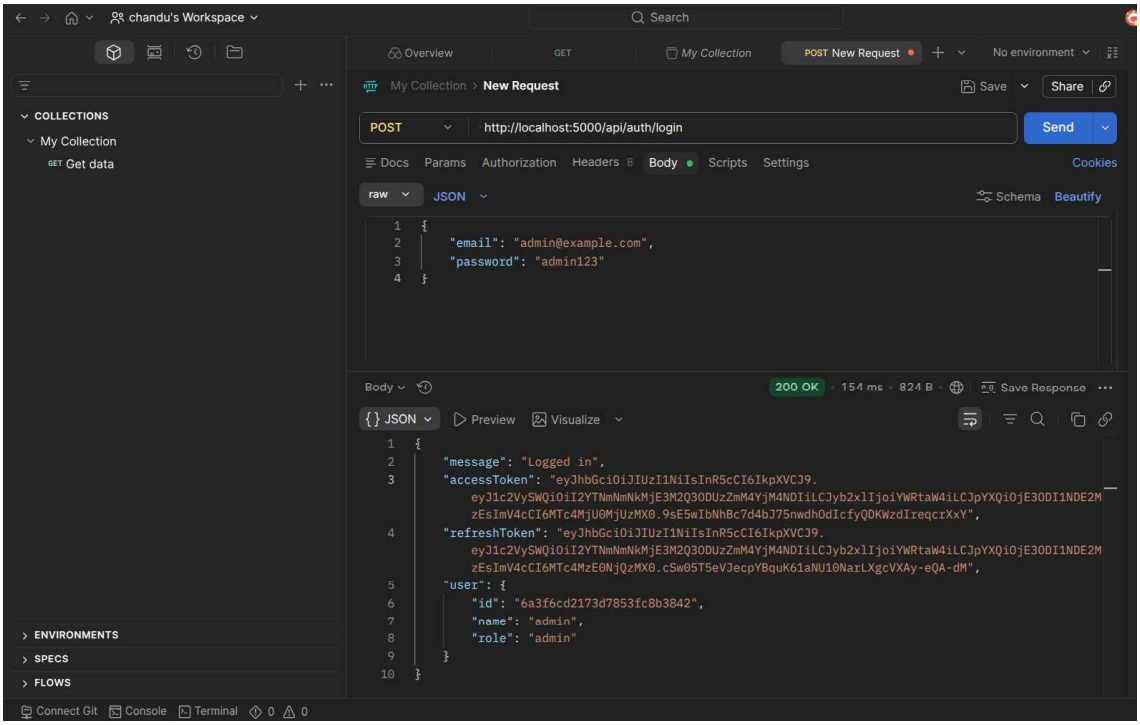


2. User Login

Checks the given credentials and returns a JWT access token if authentication is successful. The token is then used for protected API requests.

Endpoint:

POST /api/auth/login

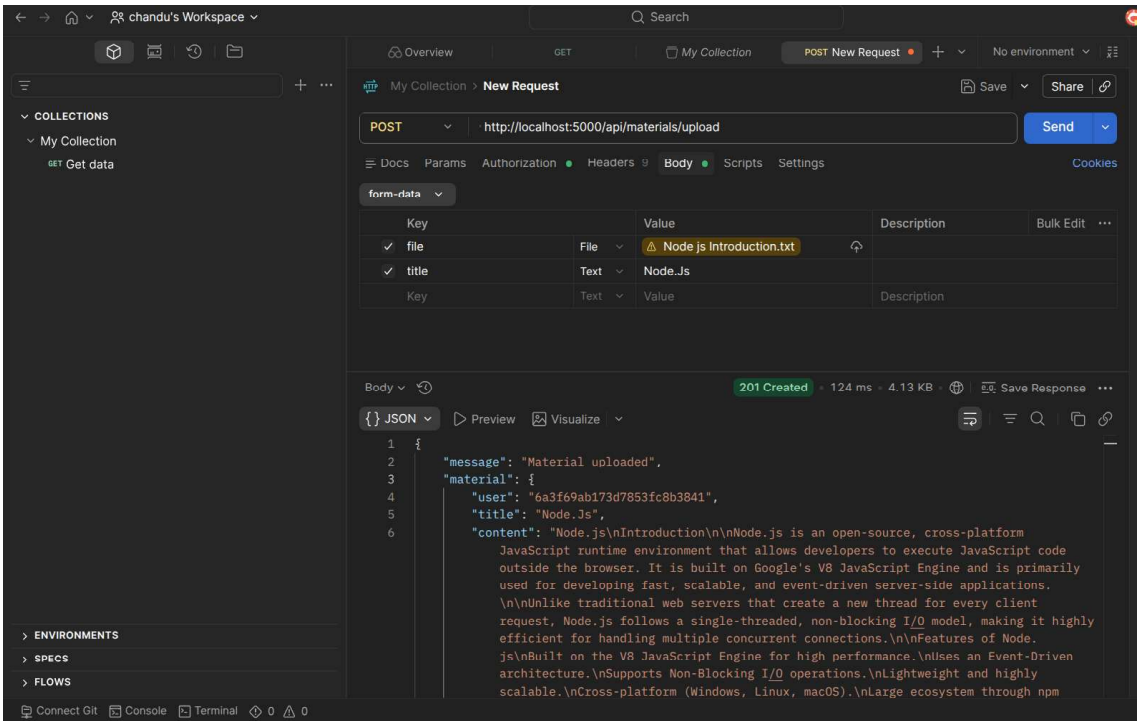


3. Upload Study Material

Lets a logged-in student upload learning material that will later be used by the AI generation services.

Endpoint:

POST /api/material/upload

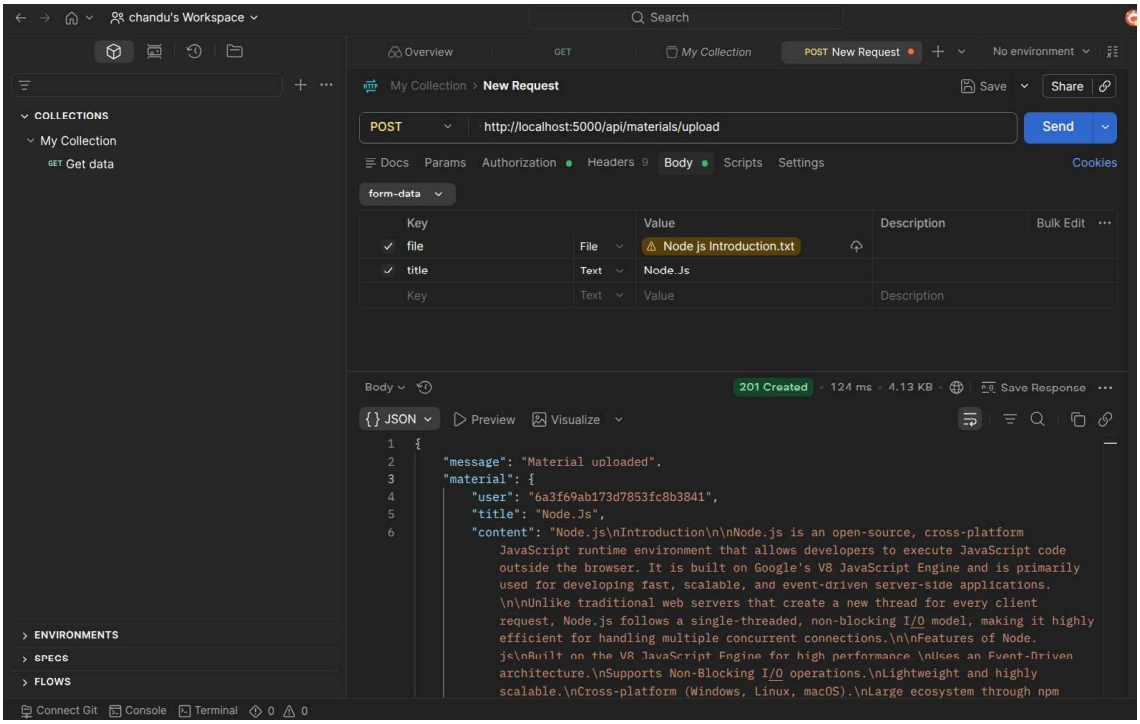


4. Generate AI Summary

Sends the selected study material to the Gemini service and returns a short AI-generated summary.

Endpoint:

POST /api/materials/:id/summarize

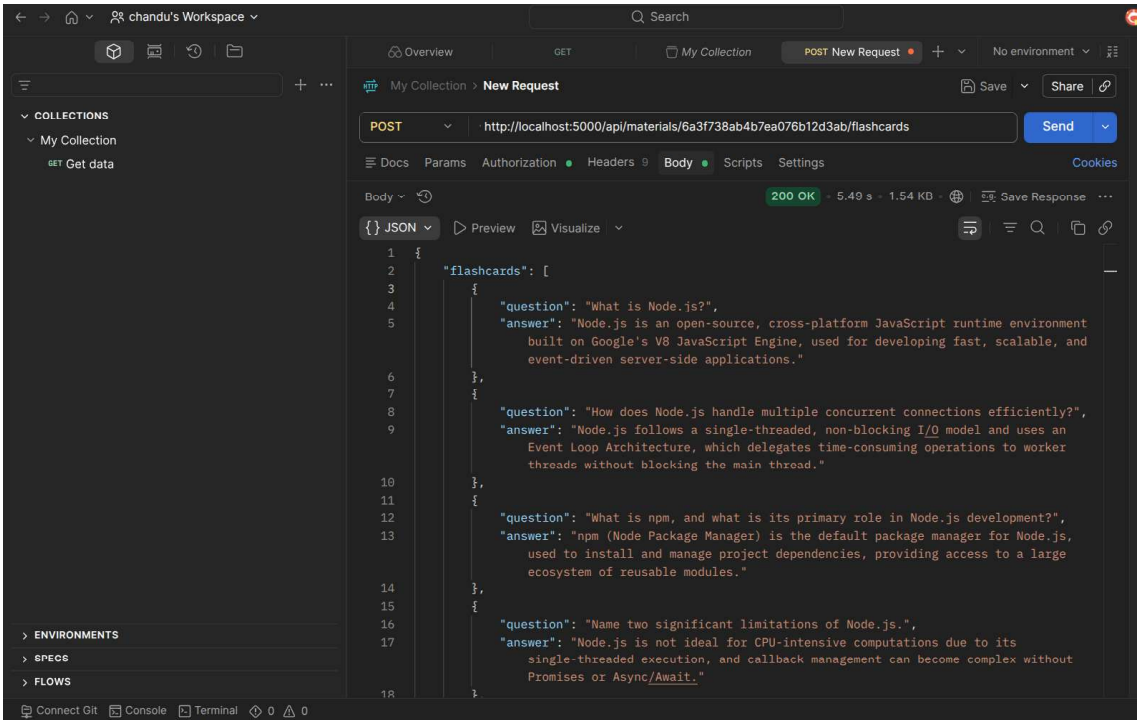


5. Generate AI Flashcards

Turns the submitted study content into AI-generated question-and-answer flashcards for revision.

Endpoint:

POST /api/ai/flashcards

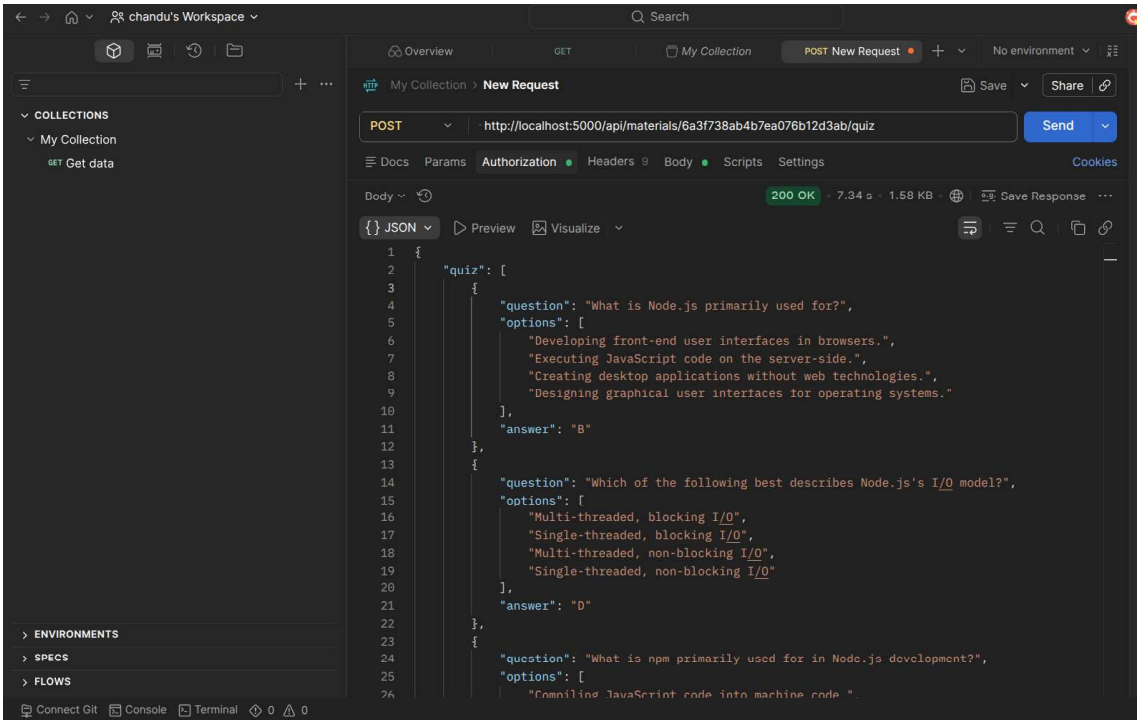


6. Generate AI Quiz

Generates multiple-choice practice questions from the selected material so the student can test their understanding.

Endpoint:

POST /api/ai/quiz

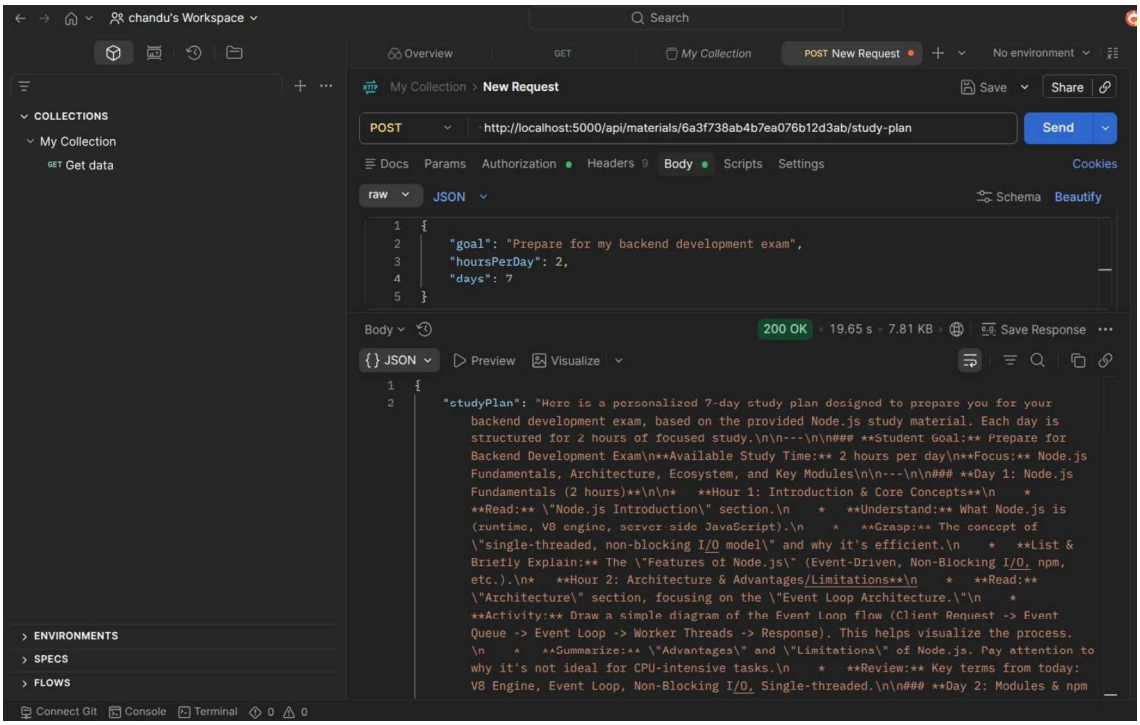


7. Generate AI Study Plan

Creates a personalised study timetable using the learner's goals, available study hours, and target examination date.

Endpoint:

POST /api/ai/study-plan


chandu's Workspace
Search
Overview
GET
My Collection
POST New Request
No environment
COLLECTIONS
My Collection
GET Get data
My Collection › New Request
Save
Share
POST
http://localhost:5000/api/materials/6a3f738ab4b7ea076b12d3ab/study-plan
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"goal": "Prepare for my backend development exam",
"hoursPerDay": 2,
"days": 7
}
Body
200 OK
19.65 s
7.81 KB
Save Response
{ } JSON
Preview
Visualize
{
"studyPlan": "Here is a personalized 7-day study plan designed to prepare you for your backend development exam, based on the provided Node.js study material. Each day is structured for 2 hours of focused study.\n\n---\n\n### **Student Goal:** Prepare for Backend Development Exam\n**Available Study Time:** 2 hours per day\n**Focus:** Node.js Fundamentals, Architecture, Ecosystem, and Key Modules\n\n---\n\n### **Day 1: Node.js Fundamentals (2 hours)**\n\n* **Hour 1: Introduction & Core Concepts**\n * **Read:** \"Node.js Introduction\" section.\n * **Understand:** What Node.js is (runtime, V8 engine, server side JavaScript).\n * **Grasp:** The concept of \"single-threaded, non-blocking I/O model\" and why it's efficient.\n * **List & Briefly Explain:** The \"Features of Node.js\" (Event-Driven, Non-Blocking I/O, npm, etc.).\n* **Hour 2: Architecture & Advantages/Limitations**\n * **Read:** \"Architecture\" section, focusing on the \"Event Loop Architecture.\"\n * **Activity:** Draw a simple diagram of the Event Loop flow (Client Request -> Event Queue -> Event Loop -> Worker Threads -> Response). This helps visualize the process.\n * **Summarize:** \"Advantages\" and \"Limitations\" of Node.js. Pay attention to why it's not ideal for CPU-intensive tasks.\n * **Review:** Key terms from today: V8 Engine, Event Loop, Non-Blocking I/O, Single-threaded.\n\n### **Day 2: Modules & npm
ENVIRONMENTS
SPECS
FLOWS
Connect Git
Console
Terminal
0
0